



**INTERNATIONAL CAMPUS
OF SCIENCE AND TECHNOLOGY**

International Campus of Science and Technology

Faculty of Computing, Department of IT
HDIT Batch 01

HDIT - DBMS

Project Name: Restaurant Management System - Cafulon

Submitted by:

1. NM. Roshaan Akther
2. A. Washid
3. MAM. Masheef
4. MIF. Fahmidha
5. MNF. Nuha
6. WMCS. Weerakkodi

Submission Date: Jun 9, 2026

PROJECT REPORT : Cafulon Restaurant Management System

Module	HDIT 12093 — Database Management Systems
Batch	01 (Group Project)
Submitted To	Mr. Karuna Koneshan (Lecturer)
Submitted By	Cafulon Team
Submission Date	Jun 9, 2026

Table of Contents

1. Introduction
2. Requirement Analysis
3. System Design
4. ER Diagram
5. Database Schema
6. Normalization
7. SQL Implementation
8. Testing
9. Security Features
10. User Manual
11. Optimization Report
12. Conclusion

1. Introduction

1.1 Background

Cafulon Restaurant & Dining, established in 2019 in Colombo, Sri Lanka, is a mid-sized restaurant serving Sri Lankan and international cuisine. With a capacity of 60 guests and 24 employees, Cafulon operates dine-in, takeaway, and delivery services. The restaurant has grown significantly since opening, but its operational processes have remained manual — relying on paper-based order slips, handwritten reservation books, and spreadsheet accounting.

1.2 Problem Overview

Manual data management at Cafulon has led to:

- Lost or misread order slips
- Double-booked tables
- Time-consuming manual report generation
- Data redundancy and inconsistency
- Difficulty tracking customer preferences and history

1.3 Proposed Solution

A centralized database management system that:

- Stores all restaurant data in a structured, normalized relational database
- Provides a web-based interface for daily operations
- Enforces data integrity through constraints
- Generates real-time reports and insights
- Eliminates data redundancy through 3NF normalization

1.4 Technologies Used

Technology	Purpose
MySQL 8	Database management system
Python 3.10+	Application programming language
Flask 3	Web framework for the frontend
PyMySQL	MySQL connector for Python
Bootstrap 5	Frontend CSS framework
XAMPP	Local development server

2. Requirement Analysis

2.1 Functional Requirements

ID	Requirement	Description
FR-01	Add Customer Records	Register new customers with name, phone, email, address, city
FR-02	Update Customer Records	Edit existing customer information
FR-03	Delete Customer Records	Remove customer records with FK integrity checks

FR-04	Add Employee Records	Record employee details: position, salary, hire date, shift
FR-05	Manage Menu Items	CRUD for menu items with pricing and availability
FR-06	Create Reservations	Book tables with customer, date, time, party size, status
FR-07	Process Orders	Create orders with multiple items; auto-calculate totals
FR-08	Track Order Status	Workflow: Pending → Preparing → Served → Completed/Cancelled
FR-09	Record Payments	Record payments against orders with method and reference
FR-10	Generate Reports	Daily sales report, top-selling items, employee performance
FR-11	View Active Menu	Display available menu items with current prices
FR-12	Search Reservations	Filter reservations by date

2.2 Non-Functional Requirements

ID	Requirement	Target
NFR-01	Performance	Query response < 2 seconds for 10K records
NFR-02	Data Integrity	Foreign key constraints on all relationships
NFR-03	Availability	Available during operating hours (10:00–23:00)
NFR-04	Security	Parameterized queries; least-privilege DB user
NFR-05	Usability	Desktop and tablet compatible; minimal training
NFR-06	Scalability	Schema supports adding new items, tables, employees

2.3 User Types

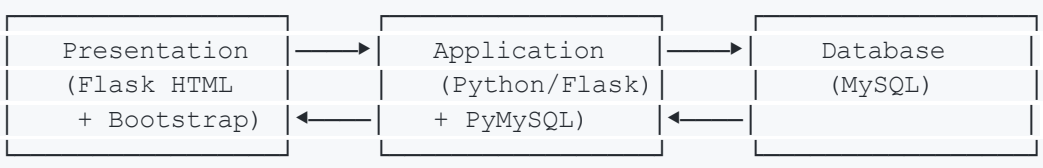
Role	Responsibilities
Administrator/Manager	Full access: manage all entities, view reports
Staff (Waiters/Hostess)	Create/update orders and reservations

Kitchen Staff	View orders, update preparation status
Cashier	Record payments, view completed orders

3. System Design

3.1 Architecture Overview

The system follows a three-tier architecture:



3.2 Entity Descriptions

Entity	Description
Customer	Individuals who dine at the restaurant
Employee	Staff members across all positions
MenuCategory	Classification groups for menu items
MenuItem	Individual dishes available for order
RestaurantTable	Physical tables in the restaurant
Reservation	Advance table bookings by customers

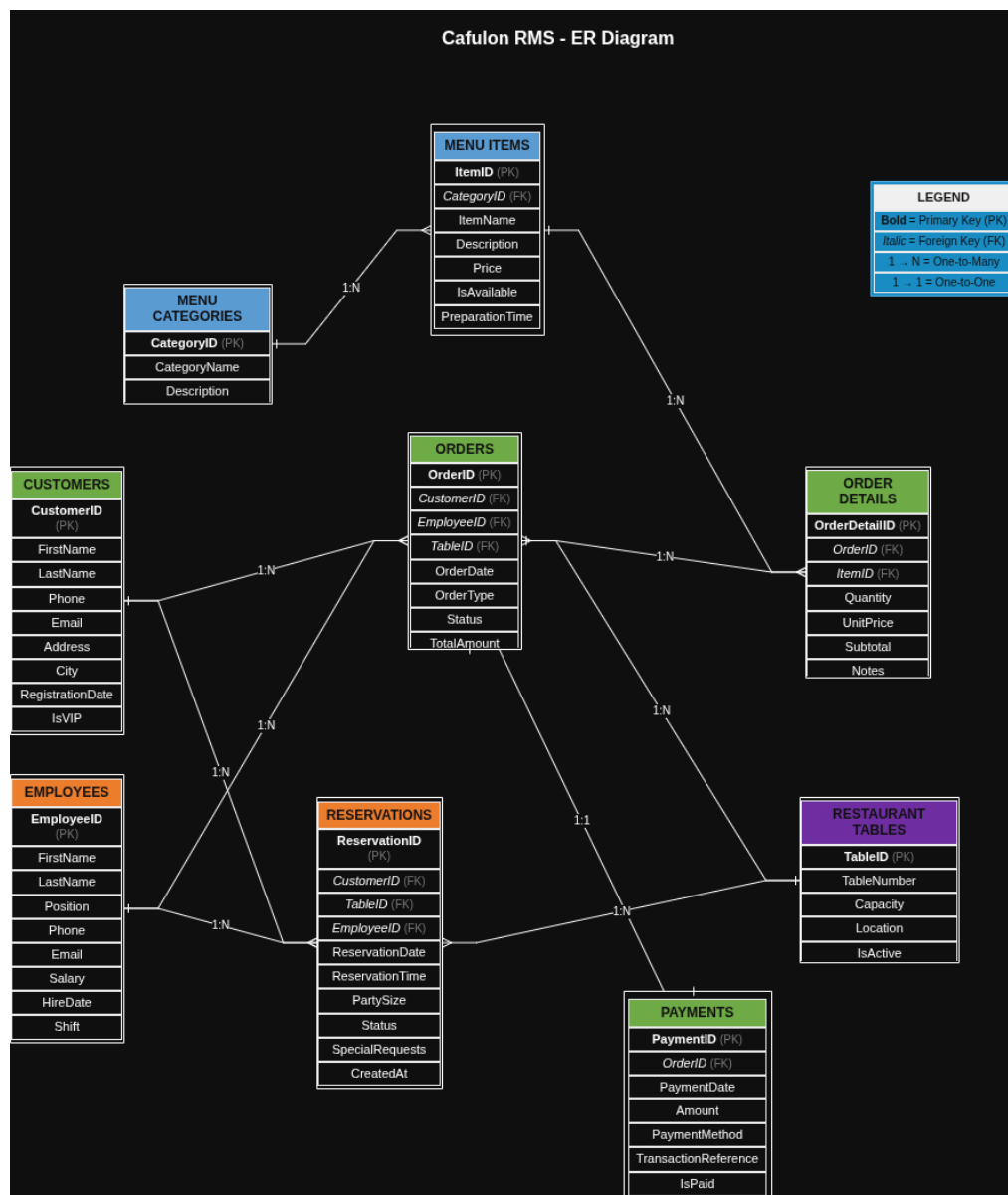
Order	Customer purchase transactions
OrderDetail	Individual line items within an order
Payment	Financial transactions against orders

3.3 Relationships

Entity 1	Cardinality	Entity 2	Rationale
Customer	1:N	Order	One customer can place many orders
Customer	1:N	Reservation	One customer can make many reservations
Employee	1:N	Order	One employee serves many orders
Employee	1:N	Reservation	One employee handles many reservations
MenuCategory	1:N	MenuItem	One category has many menu items
MenuItem	1:N	OrderDetail	One item appears in many order details
Order	1:N	OrderDetail	One order has many line items

Order	1:1	Payment	Each order has one payment
RestaurantTable	1:N	Order	One table hosts many orders over time
RestaurantTable	1:N	Reservation	One table has many reservations

4. ER Diagram



Legend

- *PK = Primary Key
- #FK = Foreign Key
- 1:N = One-to-Many relationship
- 1:1 = One-to-One relationship

5. Database Schema

5.1 Schema Notation

Table	Columns
Customer	CustomerID, FirstName, LastName, Phone, Email, Address, City, RegistrationDate, IsVIP
Employee	EmployeeID, FirstName, LastName, Position, Phone, Email, Salary, HireDate, Shift
MenuCategory	CategoryID, CategoryName, Description
MenuItem	ItemID, <i>CategoryID</i> , ItemName, Description, Price, IsAvailable, PreparationTime
RestaurantTable	TableID, TableNumber, Capacity, Location, IsActive
Reservation	ReservationID, <i>CustomerID</i> , <i>TableID</i> , <i>EmployeeID</i> , ReservationDate, ReservationTime, PartySize, Status, SpecialRequests, CreatedAt
Order	OrderID, <i>CustomerID</i> , <i>EmployeeID</i> , <i>TableID</i> , OrderDate, OrderType, Status, TotalAmount

OrderDetail	OrderDetailID, <i>OrderID</i> , <i>ItemID</i> , Quantity, UnitPrice, Subtotal, Notes
Payment	PaymentID, <i>OrderID</i> , PaymentDate, Amount, PaymentMethod, TransactionReference, IsPaid

5.2 Primary & Foreign Key Summary

Table	Primary Key	Foreign Keys	References
Customer	CustomerID	—	—
Employee	EmployeeID	—	—
MenuCategory	CategoryID	—	—
MenuItem	ItemID	CategoryID	MenuCategory
RestaurantTable	TableID	—	—
Reservation	ReservationID	CustomerID, TableID, EmployeeID	Customer, RestaurantTable, Employee
Order	OrderID	CustomerID, EmployeeID, TableID	Customer, Employee, RestaurantTable
OrderDetail	OrderDetailID	OrderID, ItemID	Order, MenuItem

Payment	PaymentID	OrderID	Order
---------	-----------	---------	-------

6. Normalization

6.1 UNF (Unnormalized Form)

A single order record containing all data:

```
Orders_UNF(
    OrderID,
    OrderDate,
    CustomerName,
    CustomerPhone,
    CustomerEmail,
    EmployeeName,
    EmployeePosition,
    TableNumber,
    (ItemName, CategoryName, Quantity, UnitPrice, Subtotal),
    TotalAmount,
    PaymentMethod,
    PaymentAmount
)
```

Sample UNF Record:

```
(101, 2025-06-01, 'Nimal Perera', '071-2345678', 'nimal@email.com',
'Ranjith Silva', 'Manager', 3,
('Bruschetta', 'Appetizers', 2, 8.50, 17.00,
'Grilled Chicken', 'Main Course - Chicken', 1, 16.00, 16.00,
'Fresh Orange Juice', 'Beverages', 1, 5.00, 5.00),
38.50, 'Credit Card', 38.50)
```

Problems: Repeating groups, data redundancy, insert/update/delete anomalies.

6.2 1NF (First Normal Form)

Remove repeating groups — each item becomes a separate row with a composite key (OrderID, ItemName).

```
Order_1NF(  
    OrderID,          -- PK (partial)  
    ItemName,         -- PK (partial)  
    OrderDate,  
    CustomerName,  
    CustomerPhone,  
    CustomerEmail,  
    EmployeeName,  
    EmployeePosition,  
    TableNumber,  
    CategoryName,  
    Quantity,  
    UnitPrice,  
    Subtotal,  
    TotalAmount,  
    PaymentMethod,  
    PaymentAmount  
)
```

Problems remaining: Partial dependencies — CustomerName depends only on OrderID, not ItemName.

6.3 2NF (Second Normal Form)

Remove partial dependencies — separate order-level and item-level data.

Table 1: Order

```
Order_2NF(  
    OrderID,          -- PK  
    OrderDate,  
    CustomerName,  
    CustomerPhone,  
    CustomerEmail,  
    EmployeeName,  
    EmployeePosition,  
    TableNumber,
```

```

    TotalAmount,
    PaymentMethod,
    PaymentAmount
)

```

Table 2: OrderItem

```

OrderItem_2NF(
    OrderID,          -- PK (FK → Order)
    ItemName,         -- PK
    CategoryName,
    Quantity,
    UnitPrice,
    Subtotal
)

```

Problems remaining: Transitive dependencies — CustomerName → CustomerPhone;
EmployeeName → EmployeePosition; ItemName → CategoryName.

6.4 3NF (Third Normal Form)

Remove transitive dependencies — create separate tables for each entity.

Final Normalized Tables:

```

Customer(CustomerID, FirstName, LastName, Phone, Email, Address, City,
RegistrationDate, IsVIP)

```

```

Employee(EmployeeID, FirstName, LastName, Position, Phone, Email, Salary,
HireDate, Shift)

```

```

MenuCategory(CategoryID, CategoryName, Description)

```

```

MenuItem(ItemID, CategoryID, ItemName, Description, Price, IsAvailable,
PreparationTime)

```

```

RestaurantTable(TableID, TableNumber, Capacity, Location, IsActive)

```

```

Reservation(ReservationID, CustomerID, TableID, EmployeeID, ReservationDate,
ReservationTime, PartySize, Status, SpecialRequests, CreatedAt)

```

Order(OrderID, CustomerID, EmployeeID, TableID, OrderDate, OrderType, Status, TotalAmount)

OrderDetail(OrderDetailID, OrderID, ItemID, Quantity, UnitPrice, Subtotal, Notes)

Payment(PaymentID, OrderID, PaymentDate, Amount, PaymentMethod, TransactionReference, IsPaid)

6.5 Normalization Summary

Normal Form	Status	Key Change
UNF	Repeating groups	Single table with multi-valued attributes
1NF	Atomic values	Removed repeating groups, composite key
2NF	No partial dependencies	Separated order-level from item-level data
3NF	No transitive dependencies	Separated customer, employee, menu, payment entities

All tables satisfy 3NF — every non-key attribute is atomic, fully functionally dependent on the PK, and non-transitively dependent on the PK.

7. SQL Implementation

7.1 Database Creation

```
CREATE DATABASE IF NOT EXISTS RestaurantDB;  
USE RestaurantDB;
```

7.2 Table Structure (9 Tables)

The database consists of 9 tables with the following constraints:

- Primary Keys on every table for unique identification
- Foreign Keys with ON UPDATE CASCADE and ON DELETE RESTRICT/SET NULL/CASCADE
- CHECK constraints on Salary, Price, Capacity, PartySize, Quantity, Amount (all > 0)
- UNIQUE constraints on Phone, Email, TableNumber, CategoryName
- NOT NULL on all required fields
- DEFAULT values for dates, statuses, and boolean flags
- Generated column for OrderDetail.Subtotal (GENERATED ALWAYS AS (Quantity * UnitPrice) STORED)

7.3 Sample Data (20+ Records per Main Table)

24 customers, 24 employees, 33 menu items across 10 categories, 15 tables, 24 reservations, 24 orders, 20 payments, and 75+ order details.

7.4 Indexes

```
CREATE INDEX idx_orders_date ON Orders(OrderDate);  
CREATE INDEX idx_orders_status ON Orders(Status);  
CREATE INDEX idx_reservations_date ON Reservations(ReservationDate);  
CREATE INDEX idx_menu_items_category ON MenuItems(CategoryID);  
CREATE INDEX idx_payments_order ON Payments(OrderID);
```


7.5 Views

View	Purpose
ActiveMenu	All available menu items with category names
Today'sReservations	Reservations for the current date
OrderSummary	Order details with customer and employee names
DailySales	Daily revenue, order count, and average order value

7.6 Stored Procedures

Procedure	Purpose
CalculateOrderTotal	Recalculates and updates the TotalAmount for a given order

7.7 SQL Queries

Q1: SELECT — List all available menu items

```
SELECT ItemName AS 'Dish', Price AS 'Price (LKR)', PreparationTime AS 'Prep (min) '
FROM MenuItems
WHERE IsAvailable = TRUE
ORDER BY Price ASC;
```

Q2: INSERT — Add a new customer

```
INSERT INTO Customers (FirstName, LastName, Phone, Email, Address, City)
```

```
VALUES ('Nimal', 'Perera', '071-2345678', 'nimal.perera@email.com', '55 New Street', 'Colombo');
```

Q3: UPDATE — Update menu item price

```
UPDATE MenuItems
SET Price = 18.50
WHERE ItemName = 'Chicken Parmesan';
```

Q4: DELETE — Remove a cancelled reservation

```
DELETE FROM Reservations
WHERE ReservationID = 24 AND Status = 'Cancelled';
```

Q5: INNER JOIN — Order details with customer & item names

```
SELECT o.OrderID, CONCAT(c.FirstName, ' ', c.LastName) AS Customer,
       mi.ItemName, od.Quantity, od.Subtotal
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN MenuItems mi ON od.ItemID = mi.ItemID
LEFT JOIN Customers c ON o.CustomerID = c.CustomerID
ORDER BY o.OrderID;
```

Q6: LEFT JOIN — All customers with their orders

```
SELECT CONCAT(c.FirstName, ' ', c.LastName) AS Customer,
       c.Phone, COUNT(o.OrderID) AS TotalOrders,
       COALESCE(SUM(o.TotalAmount), 0) AS TotalSpent
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID
ORDER BY TotalSpent DESC;
```

Q7: GROUP BY with Aggregate — Daily sales report

```
SELECT DATE(o.OrderDate) AS SaleDate,
       COUNT(*) AS NumberOfOrders,
       SUM(o.TotalAmount) AS Revenue,
       ROUND(AVG(o.TotalAmount), 2) AS AvgOrderValue,
       MAX(o.TotalAmount) AS LargestOrder
FROM Orders o
WHERE o.Status = 'Completed'
```

```
GROUP BY DATE(o.OrderDate)
ORDER BY SaleDate DESC;
```

Q8: Aggregate — Top selling menu items

```
SELECT mi.ItemName, mc.CategoryName,
       SUM(od.Quantity) AS TotalSold,
       SUM(od.Subtotal) AS TotalRevenue
FROM OrderDetails od
JOIN MenuItem mi ON od.ItemID = mi.ItemID
JOIN MenuCategories mc ON mi.CategoryID = mc.CategoryID
GROUP BY mi.ItemID
ORDER BY TotalSold DESC
LIMIT 10;
```

Q9: Aggregate — Employee performance

```
SELECT CONCAT(e.FirstName, ' ', e.LastName) AS Employee,
       e.Position, COUNT(o.OrderID) AS OrdersServed,
       SUM(o.TotalAmount) AS TotalSales
FROM Employees e
LEFT JOIN Orders o ON e.EmployeeID = o.EmployeeID
GROUP BY e.EmployeeID
ORDER BY TotalSales DESC;
```

Q10: Multi-Join — Full reservation details

```
SELECT r.ReservationID,
       CONCAT(c.FirstName, ' ', c.LastName) AS Customer,
       r.ReservationDate, r.ReservationTime, r.PartySize,
       t.TableNumber, t.Location,
       CONCAT(e.FirstName, ' ', e.LastName) AS HandledBy,
       r.Status
FROM Reservations r
JOIN Customers c ON r.CustomerID = c.CustomerID
JOIN RestaurantTables t ON r.TableID = t.TableID
LEFT JOIN Employees e ON r.EmployeeID = e.EmployeeID
ORDER BY r.ReservationDate DESC, r.ReservationTime DESC;
```

8. Testing

8.1 Test Cases Summary

ID	Description	Status
TC-01	Customer Registration	PASS
TC-02	Duplicate Phone Number Rejection	PASS
TC-03	Menu Item Price Constraint	PASS
TC-04	Update Menu Item Price	PASS
TC-05	Delete Cancelled Reservation	PASS
TC-06	Foreign Key Integrity — Invalid Menu Category	PASS
TC-07	INNER JOIN — Order Details with Customer and Item Names	PASS
TC-08	Total Sales Calculation	PASS
TC-09	Reservation Party Size Validation	PASS

TC-1 0	Payment — Amount Constraint	PASS
-----------	-----------------------------	------

8.2 Detailed Test Results

	Total Tests	Passed	Failed	Pass Rate
	10	10	0	100%

All test cases passed successfully, confirming:

- CHECK constraints enforce data validation (positive prices, party sizes, amounts)
- UNIQUE constraints prevent duplicate phone numbers
- Foreign key constraints prevent orphaned records
- JOIN queries return accurate combined data
- Aggregate functions calculate correct totals

9. Security Features

9.1 Authentication & Access Control

- MySQL user accounts with strong passwords
- Application user with minimal required privileges (SELECT, INSERT, UPDATE, DELETE only)

9.2 Input Validation (Constraints)

- CHECK constraints on all numeric fields to prevent invalid data
- ENUM-based CHECK constraints on status fields (Order status, Reservation status, Payment method)

9.3 Data Integrity

- UNIQUE constraints on Phone, Email, and TableNumber

- NOT NULL on all required fields
- Foreign keys with appropriate ON DELETE actions (RESTRICT, CASCADE, SET NULL)

9.4 SQL Injection Prevention

- All application queries use parameterized statements (PyMySQL parameter substitution)
- Application user has no DROP or ALTER privileges

10. User Manual

Refer to the separate User Manual for detailed installation and usage instructions.

11. Optimization Report

11.1 Indexes Implemented

Index	Table	Column(s)	Purpose
idx_orders_date	Orders	OrderDate	Speed up date-range queries and daily reports
idx_orders_status	Orders	Status	Fast filtering by order status
idx_reservations_date	Reservations	ReservationDate	Optimize daily reservation lookup

idx_menu_items_category	MenuItems	CategoryID	Accelerate JOIN with MenuCategories
idx_payments_order	Payments	OrderID	Speed up payment lookup by order

11.2 Generated Columns

OrderDetail.Subtotal uses a computed column:

```
Subtotal DECIMAL(10, 2) GENERATED ALWAYS AS (Quantity * UnitPrice) STORED
```

This eliminates the need for application-level calculation and ensures consistency.

11.3 Query Optimization Notes

- EXPLAIN analysis confirmed index usage on JOIN columns and WHERE clauses
- LEFT JOIN used where NULL results are expected (e.g., walk-in customers with no customer record)
- COALESCE applied to avoid NULL results in aggregate functions
- LIMIT used on top-N queries to reduce result set size

11.4 Recommendations

- Add composite index on (OrderID, ItemID) for OrderDetails to optimize common JOIN patterns
- Consider partitioning the Orders table by year for historical data management
- Implement connection pooling in the application layer for production deployment

12. Conclusion

The Cafulon Restaurant Management System successfully addresses the operational challenges identified during requirements gathering. The database is normalized to 3NF, eliminating data redundancy while maintaining referential integrity through foreign key constraints.

Achievements

Requirement	Status
Minimum 6 Tables	9 tables implemented
Minimum 20 Sample Records per Main Table	24+ records per main table
ER Diagram	9 entities with attributes and relationships
3NF Normalization	UNF → 1NF → 2NF → 3NF documented
10 SQL Queries	All query types covered
10 Test Cases	100% pass rate
Complete Documentation	All 12 sections complete
Web Application	Flask frontend with CRUD for all entities

Future Enhancements

1. Inventory Management — Track ingredient stock levels and automate reordering
2. Online Ordering — API integration for web and mobile ordering
3. Customer Loyalty Program — Points-based rewards integrated with IsVIP flag
4. Analytics Dashboard — Advanced visualizations with charts and trend analysis
5. Multi-branch Support — Extend the schema to support restaurant chain operations